

CSS4003 — DevOps 2

Lecture 6: Introduction to Cloud & AWS

Adil Akhmetov · Lesson 6



Recap — Lesson 5 (Advanced Git & GitHub)

- Why create a hotfix branch from `main` , not from `develop` ? (`main` is what's in production — `develop` carries unfinished work you don't want to ship with the fix)
- A commit pushed to a shared branch broke things. `git revert` or `git reset` ? (`revert` — it adds a new inverse commit instead of rewriting history other people already pulled)
- Three local commits vanished after `git reset --hard HEAD~3` . Where do you look? (`git reflog` — it records where `HEAD` has been, including the "lost" commits)

Today's agenda

- ☐ What "cloud" actually means — and what it doesn't
- ☐ Service models (IaaS / PaaS / SaaS) and deployment models
- ☐ The shared responsibility model
- ☐ AWS global infrastructure: regions, Availability Zones, edge
- ☐ A tour of the core services we'll use in weeks 7–9
- ☐ Keeping your account safe and your bill at zero
- ☐ Console vs CLI → straight into today's practice

The scenario

Your team's app from the Git weeks is ready to go live. The old plan: buy a server, wait two weeks for delivery, rack it, install Linux, and hope it's big enough. A classmate says "just put it on AWS". Another one opened an AWS account last year, left a server running, and got a bill.

Today is about both halves of that story: why the cloud is the default answer, and how to use it without getting burned.

Block 1

What cloud computing is

Cloud = someone else's data center, rented by API

- You don't buy hardware. You **request** compute, storage, or a database — from a web console, a CLI, or code — and get it in minutes.
- You pay for what you use, while you use it. Turn it off, the meter stops (mostly — we'll get to the exceptions).
- The provider owns the buildings, power, cooling, networking, and physical servers.

"The cloud" isn't magic and isn't weather. It's real servers in real buildings — you just never touch them.

The NIST definition: five essential characteristics

Characteristic	In plain words
On-demand self-service	You provision it yourself, no ticket to a human
Broad network access	Reachable over the network from standard clients
Resource pooling	Many customers share the same physical hardware
Rapid elasticity	Scale up and down quickly, sometimes automatically
Measured service	Usage is metered — and billed

Source: NIST SP 800-145. A rented server you have to email someone about isn't "cloud" by this definition — self-service and metering are the point.

CapEx vs OpEx

On-premises — CapEx

- Pay up front for hardware
- Guess capacity years ahead
- Over-buy → idle servers
- Under-buy → outage on launch day

Cloud — OpEx

- Pay monthly for what you use
- Start small, grow when needed
- Experiments are cheap to throw away
- But: forgotten resources keep billing

OpEx is a double-edged sword. Nobody approves a cloud bill in advance — it just arrives. That's why Block 4 exists.

Service models: who manages what

Layer	On-prem	IaaS	PaaS	SaaS
Application	You	You	You	Provider
Data	You	You	You	Provider*
Runtime / middleware	You	You	Provider	Provider
OS	You	You	Provider	Provider
Virtualization / servers	You	Provider	Provider	Provider
Network / building	You	Provider	Provider	Provider

* With SaaS you still own **your** data and who can access it — the provider runs everything underneath.

Service models: examples

- **IaaS** — you get a virtual machine, you install everything on it. AWS EC2, Google Compute Engine, Azure VMs.
- **PaaS** — you bring code or a container, the platform runs it. AWS Elastic Beanstalk, Heroku, Google App Engine.
- **SaaS** — you just use the finished product. Gmail, GitHub, Google Docs, Slack.

The trade-off is always the same: the higher you go, the less you manage — and the less you control.

Deployment models

- **Public cloud** — shared provider infrastructure (AWS, Azure, Google Cloud). What this course uses.
- **Private cloud** — cloud-style self-service, but on infrastructure dedicated to one organization (often on-prem, e.g. OpenStack).
- **Hybrid cloud** — on-prem and public cloud connected and used together. Common in banks and government.
- **Multi-cloud** — more than one public provider, on purpose.

NIST also lists a **community** cloud — shared by organizations with common requirements. You'll see it in textbooks more than in job ads.

The shared responsibility model

AWS: security **of** the cloud

- Data centers and physical security
- Hardware, network, hypervisor
- The managed services themselves

You: security **in** the cloud

- Who can log in (IAM, MFA)
- Firewall rules (security groups)
- OS patches on your EC2 instances
- Your data: encryption, backups, public or not

Most cloud breaches in the news are on the customer side: a public S3 bucket, a leaked access key, a wide-open firewall rule.

The line moves with the service model

- **EC2 (IaaS)** — you patch the OS, configure the firewall, install the runtime. A lot is yours.
- **RDS (managed database)** — AWS patches the database engine and OS; you still own users, network access, and backups settings.
- **S3 (managed storage)** — AWS runs the storage; you decide who can read each bucket.

"It's managed" never means "it's secured for me". Access control is always your job.

Quick check

Your EC2 server gets hacked through an unpatched package in Ubuntu. Is that AWS's fault or yours?

Yours — on IaaS, the guest operating system and everything installed on it are the customer's side of the shared responsibility model.

Block 2

AWS global infrastructure

Regions

- A **region** is a separate geographic area — e.g. `eu-central-1` (Frankfurt), `ap-south-1` (Mumbai), `us-east-1` (N. Virginia).
- Regions are isolated from each other. Resources you create live in **one** region unless you copy them.
- Most services are regional: an EC2 instance in Frankfurt doesn't show up when the console is set to Mumbai.

"Where did my server go?" — 9 times out of 10, the console's region selector (top-right) is set to a different region.

Availability Zones

- Each region has several **Availability Zones (AZs)** — usually three or more — named like `eu-central-1a` , `eu-central-1b` , `eu-central-1c` .
- An AZ is one or more data centers with independent power, cooling, and networking, physically separated from the other AZs.
- AZs in a region are linked with low-latency private networking.

Run two copies of your app in two AZs, and a fire or power failure in one building doesn't take you down. That's the basis of high availability.

Edge locations

- Separate from regions: a large network of smaller sites close to users.
- Used by **CloudFront** (CDN — caches your static content near users) and **Route 53** (DNS).
- You don't run servers there. They speed up delivery of what lives in your region.

Region ⊃ Availability Zones ⊃ data centers

Edge locations: separate, many, close to users

Choosing a region: four questions

1. **Latency** — how far is it from your users?
2. **Compliance** — does the law or the customer require data to stay in a specific country?
3. **Service availability** — not every service or instance type exists in every region.
4. **Price** — the same service can cost different amounts in different regions.

Check the current list and prices yourself: the AWS Global Infrastructure page and the per-service pricing pages. They change — don't trust old slides, including these.

What about Kazakhstan?

- There is **no AWS region in Kazakhstan** or elsewhere in Central Asia (as of this lecture).
- Regions people here commonly pick: **Frankfurt** (`eu-central-1`), **Stockholm** (`eu-north-1`), **Mumbai** (`ap-south-1`), and the Middle East regions (`me-central-1` UAE, `me-south-1` Bahrain).
- Some newer regions are **opt-in** — disabled until you enable them in account settings.
- Data-localization rules for personal data can force some systems to stay in local data centers — that's a compliance question, not a technical one.

For this course: use `eu-central-1` (Frankfurt) unless told otherwise, so everyone's screenshots look the same.

Quick check

You run your app on one EC2 instance in `eu-central-1a` . The whole AZ loses power. What would have kept the app up?

A second instance in another AZ of the same region (e.g. `eu-central-1b`) behind a load balancer — AZs fail independently.

Block 3

A tour of the core services

The five services this course is built on

Service	What it is	Week
 IAM	Users, groups, roles, permissions	6 (today)
 EC2	Virtual machines	7
 VPC	Your private network in AWS	8
 S3	Object storage (files, backups, static sites)	9
 RDS	Managed relational databases	9

AWS has hundreds of services. You need about five to deploy a real app — start there and ignore the rest of the console for now.

IAM — who can do what

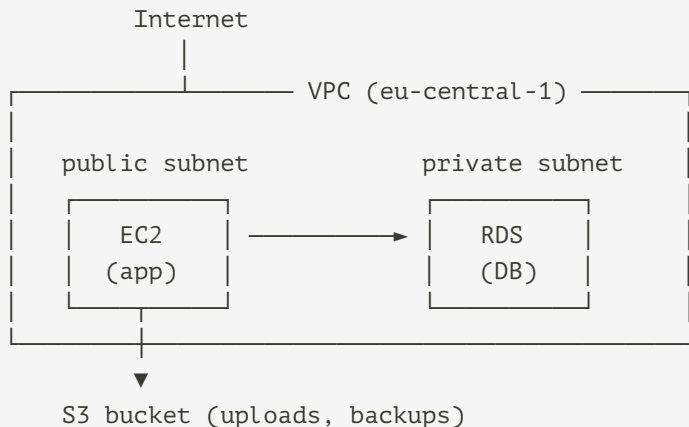
- **Users** — a person or app with long-term credentials.
- **Groups** — attach permissions once, add users to the group.
- **Roles** — temporary permissions an EC2 instance, a service, or a person can *assume*. Preferred over long-lived keys.
- **Policies** — JSON documents that allow or deny actions on resources.
- IAM is **global** — not tied to a region.

Rule of thumb: **least privilege** — give exactly the permissions a task needs, nothing more.

EC2, VPC, S3, RDS — one sentence each

- **EC2** — rent a Linux or Windows VM by the second/hour; you pick size, OS image (AMI), disk, and firewall rules (security groups).
- **VPC** — an isolated virtual network: subnets, route tables, internet gateway. Every EC2 instance lives in one.
- **S3** — store any number of files ("objects") in "buckets"; private by default. Great for backups and static websites.
- **RDS** — PostgreSQL/MySQL/etc. where AWS handles installs, patching, and automated backups.

How they fit together



IAM decides who/what may touch each of these

By week 9 you'll have built exactly this.

Block 4

Account safety & cost control

The root user

- The email you signed up with is the **root user** — it can do *everything*, including closing the account. Its permissions can't be limited inside the account.
- Use it only for a handful of account-level tasks (billing settings, closing the account).
- **Enable MFA on root first** — before anything else. AWS now requires MFA for root on many accounts; do it before you're forced to.
- Never create access keys for the root user.

Day-one checklist

1. Enable **MFA on the root user**
2. Create an **admin identity** for daily work (an IAM user in an `Admins` group, or IAM Identity Center) — and enable MFA on it too
3. Sign out of root. Sign in as the admin identity from now on
4. Create a **budget** in AWS Budgets — e.g. \$1/month with an email alert
5. Pick your region (`eu-central-1`) and stay in it

This is literally today's practice, Track A.

Free tier pitfalls

- AWS changed its free tier for **new accounts created after July 15, 2025**: a free plan with sign-up credits for a limited period instead of the old 12-month offers. Read the current terms on `aws.amazon.com/free`.
- "Free" usually has limits: instance size, hours per month, GB stored.
- Things that bill even when "nothing is running":
 - **Stopped** EC2 instances still pay for their EBS disks
 - Elastic IPs, NAT Gateways, load balancers, snapshots
 - Resources left in **another region** you forgot about

Stop vs terminate vs delete

Action	What happens	Still billing?
Stop an EC2 instance	VM shuts down, disk kept	Yes — for the disk
Terminate an instance	VM and (by default) root disk deleted	No
Delete an S3 bucket	Bucket must be empty first	No
Leave it "for later"	Nothing happens	Yes

End every practice session with cleanup. Check every region you used.

Budgets and the Pricing Calculator

- **AWS Budgets** — set a monthly amount; get an email when actual or forecasted spend crosses a threshold. A budget is an alarm, **not** a hard limit — AWS won't stop your resources for you.
- **Billing / Cost Explorer** — see what you actually spent, by service and by region.
- **AWS Pricing Calculator** (`calculator.aws`) — estimate monthly cost *before* you build. Works without an account or login.

Habit for the rest of the course: estimate first, build second, check the bill third.

Quick check

You set a \$1 budget. You forget an instance running and usage reaches \$5.
Does AWS stop the instance at \$1?

No — a basic budget only sends alerts. Stopping things is still your job (budget *actions* can automate some responses, but you have to set them up).

Block 5

Console vs CLI

Two ways to drive AWS

Console (web UI)

- Great for exploring and learning
- Shows what options exist
- Hard to repeat exactly
- Screenshots are your only record

CLI (`aws ...`)

- Repeatable — commands can be scripted
- Reviewable — commands can go in Git
- Same API the console uses
- The step before Terraform (week 14)

DevOps habit: learn in the console, then do it again from the CLI.

Setting up the CLI

```
aws --version                # AWS CLI v2
aws configure --profile css4003
# AWS Access Key ID [None]: AKIA...
# AWS Secret Access Key [None]: ****
# Default region name [None]: eu-central-1
# Default output format [None]: json
aws sts get-caller-identity --profile css4003
```

- Credentials go to `~/.aws/credentials`, settings to `~/.aws/config`.
- `sts get-caller-identity` answers "who am I logged in as?" — run it first whenever something says "access denied".

Access keys are passwords. Never commit `~/.aws/credentials` — the same "secret in history" problem from last week, with a bill attached.

Profiles

```
aws s3 ls --profile css4003
export AWS_PROFILE=css4003          # use it for the whole shell session
aws ec2 describe-regions --query 'Regions[].RegionName' --output text
aws ec2 describe-availability-zones --region eu-central-1 \
  --query 'AvailabilityZones[].ZoneName' --output text
```

- A **profile** is a named set of credentials + region. Keep one per account (personal, course, work).
- `--query` filters the JSON output (JMESPath); `--output text|table|json` changes the format.

Block 6

Today's practice

Today's practice — First steps in AWS

Track A — you have an AWS account

1. Root MFA, admin IAM user + MFA
2. \$1 monthly budget with an email alert
3. CLI profile + `get-caller-identity`
4. List regions and AZs from the CLI
5. Clean up

Track B — no AWS account

1. Pricing Calculator estimate (no login)
2. AWS CLI against a local mock (moto)
3. Same CLI moves: identity, regions, IAM user, budget
4. Service-model and shared-responsibility tasks

Either track: submit a report with commands, outputs, screenshots, and answers. Instructions in `practices/06-intro-cloud-aws/`.

By the end of this lesson, you should be able to

- ☐ Explain cloud computing using the five NIST characteristics
- ☐ Classify a service as IaaS, PaaS, or SaaS and say who manages what
- ☐ Draw the line between AWS's and your responsibilities for EC2, RDS, S3
- ☐ Explain regions, AZs, and edge locations, and justify a region choice
- ☐ Secure a new AWS account and set a budget alert
- ☐ Configure an AWS CLI profile and check who you're logged in as

Before next lecture

- ☐ Finish today's practice if you didn't wrap it up in session
- ☐ Track A: double-check every region for leftover resources
- ☐ Install the AWS CLI v2 if you haven't — next week is all EC2

Bring a laptop with a working terminal and SSH client next week.

Next lecture

AWS EC2 — launching, connecting to, and securing your first virtual machine.